ADAPTER
DESIGN PATTERN



Creational (5)

-  Factory
- Abstract Factory
-  Singleton
- Prototype
- Builder

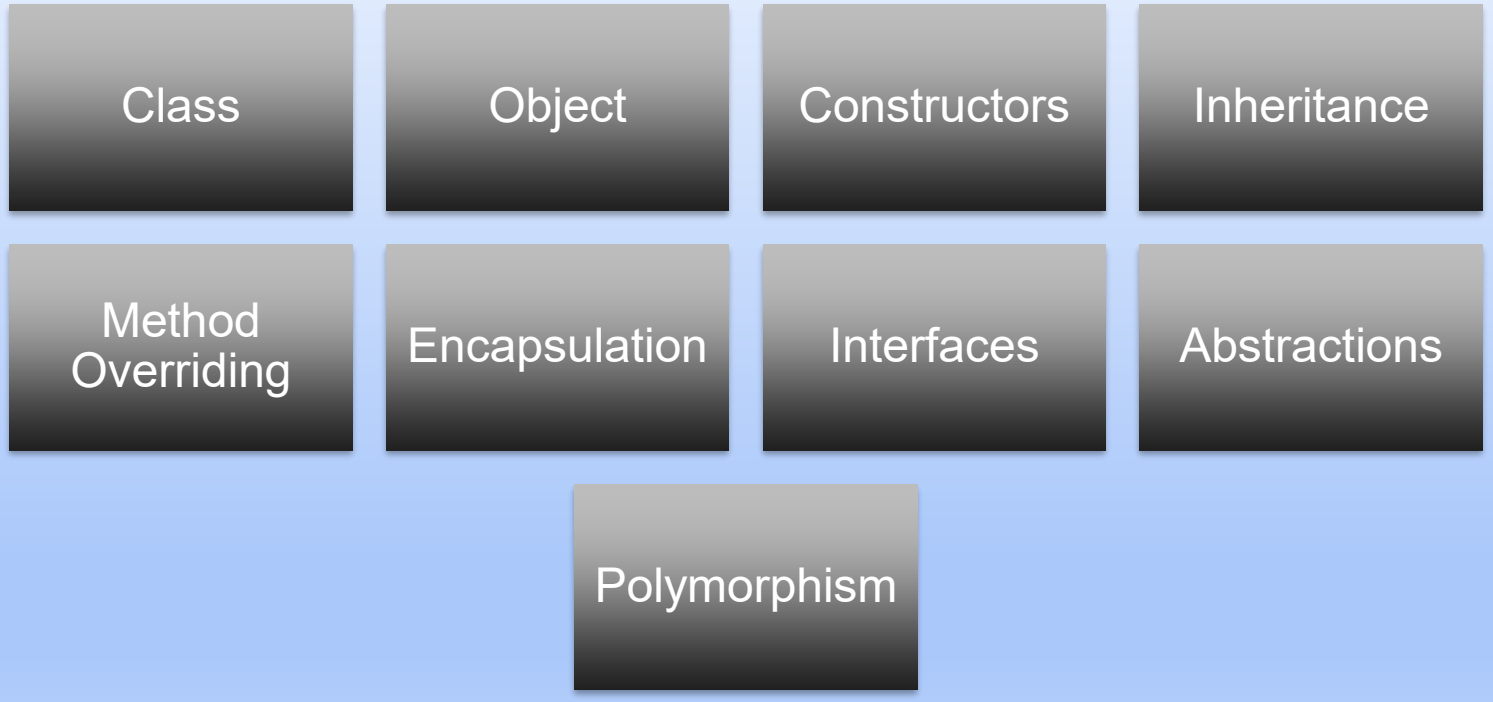
Structural (7)

-  Adapter
- Bridge
- Composite
- Decorator
-  Facade
- Flyweight
- Proxy

Behavioral (11)

- Chain Of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template
- Visitor

OOP Principals




Ringke
Ringke


plugable









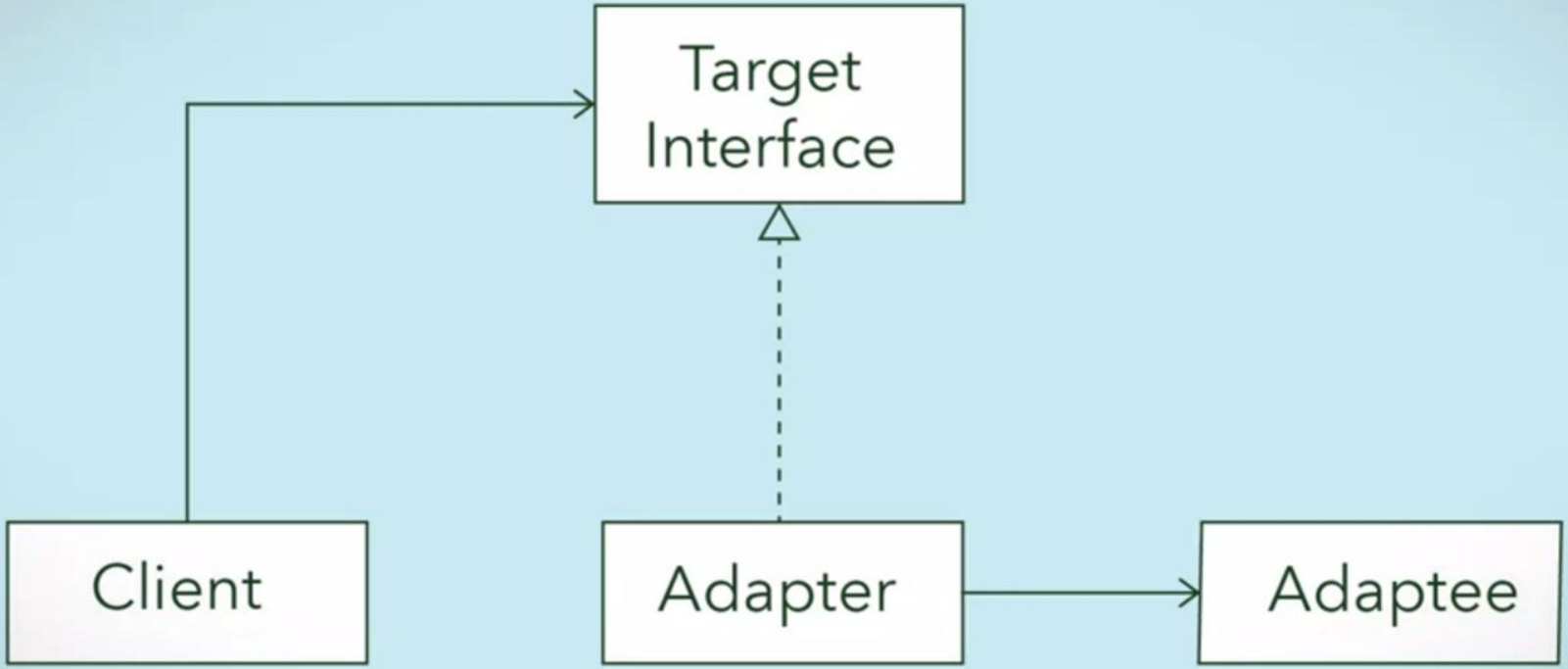
- Adapter Design pattern is used to communicate between two non-compatible existing systems by providing a compatible interface.
- Common term is wrapper class.



IF THE INTERFACES ARE INCOMPATIBLE, WHY WE CAN'T CHANGE THEM.



WHY WE CAN’T CHANGE THE CLIENT SO THAT IS COMPATIBLE WITH THE WEB SERVICE.


Target
Interface
Client
Adapter
Adaptee

Implementations



Third party libraries



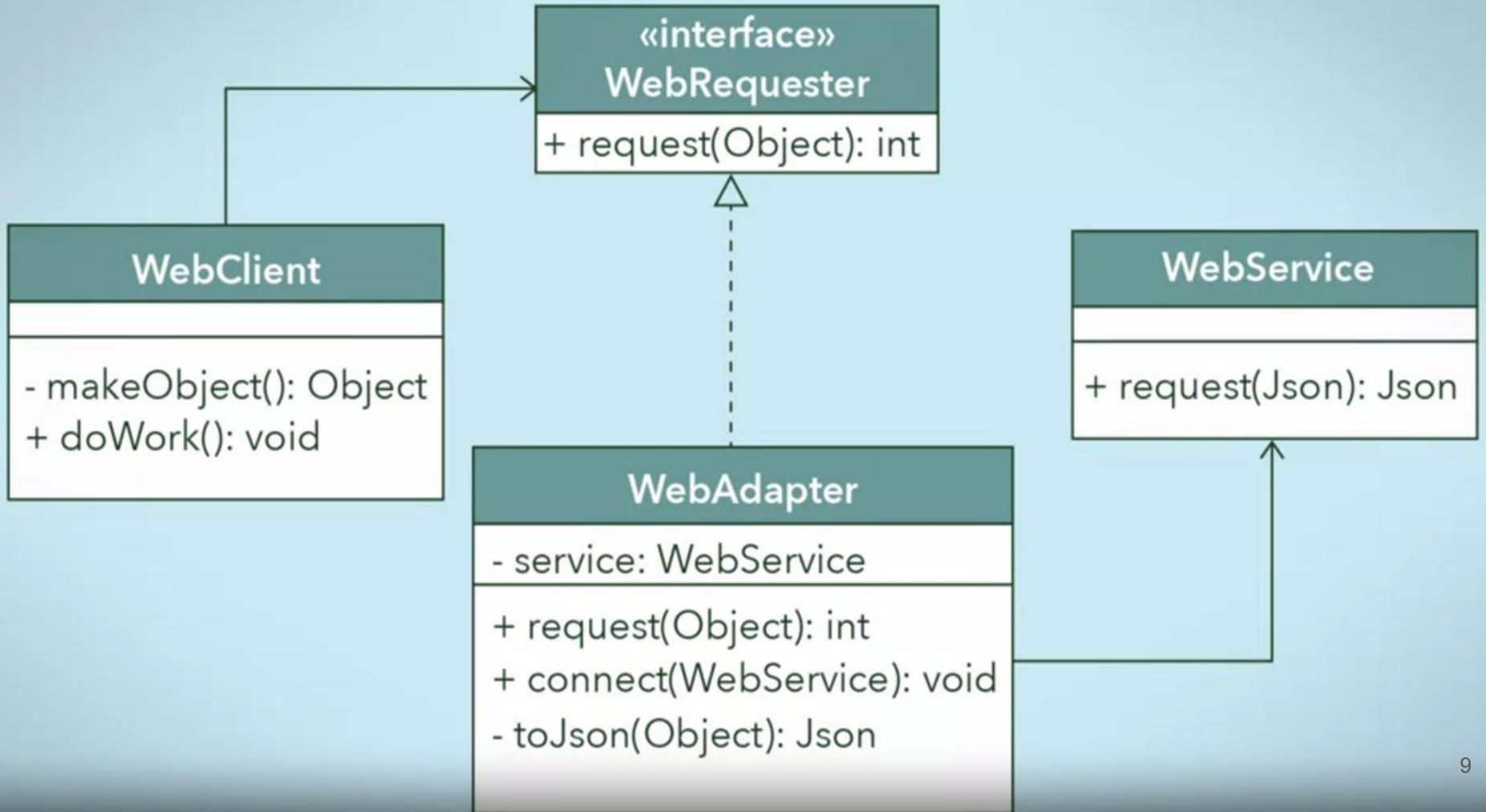
System Integration



Web Services



System Upgrades


«interface»
WebRequester
+ request(Object): int
WebClient
- makeObject(): Object
+ doWork(): void
WebService
+ request(Json): Json
WebAdapter
- service: WebService
+ request(Object): int
+ connect(WebService): void
- toJson(Object): Json

Step 1: Design the target interface

```
public interface WebRequester {
     public int request(Object);
}
```

Step 2: Implement the target interface with the adapter class

```
public class WebAdapter implements WebRequester {
     private WebService service;

     public void connect(WebService currentService) {
          this.service = currentService;
     }

     public int request(Object request) {
          Json result = this.toJson(request);
          Json response = service.request(result);
          if (response != null)
               return 200; // OK status code
          return 500; // Server error status code
     }

     private Json toJson(Object input) { … }
}
```

Step 3: Send the request from the client to the adapter using the target interface

```
public class WebClient {
     private WebRequester webRequester;

     public WebClient(WebRequester webRequester) {
          this.webRequester = webRequester;
     }

     private Object makeObject() { … } // Make an Object

     public void doWork() {
          Object object = makeObject();
          int status = webRequester.request(object);

          if (status == 200) {
               System.out.println("OK");
          } else {
               System.out.println("Not OK");
          }
          return;
     }
}
```

```
public class Program {
    public static void main(String args[]) {
        String webHost = “Host: https://google.com\n\r";
        WebService service = new WebService(webHost);
        WebAdapter adapter = new WebAdapter();
        adapter.connect(service);
        WebClient client = new WebClient(adapter);
        client.doWork();
    }
}
```

What are the characteristics of the adapter design pattern?



The client and adaptee classes have incompatible interfaces.



An adapter is a wrapper class that wraps the adaptee, hiding it from the client.

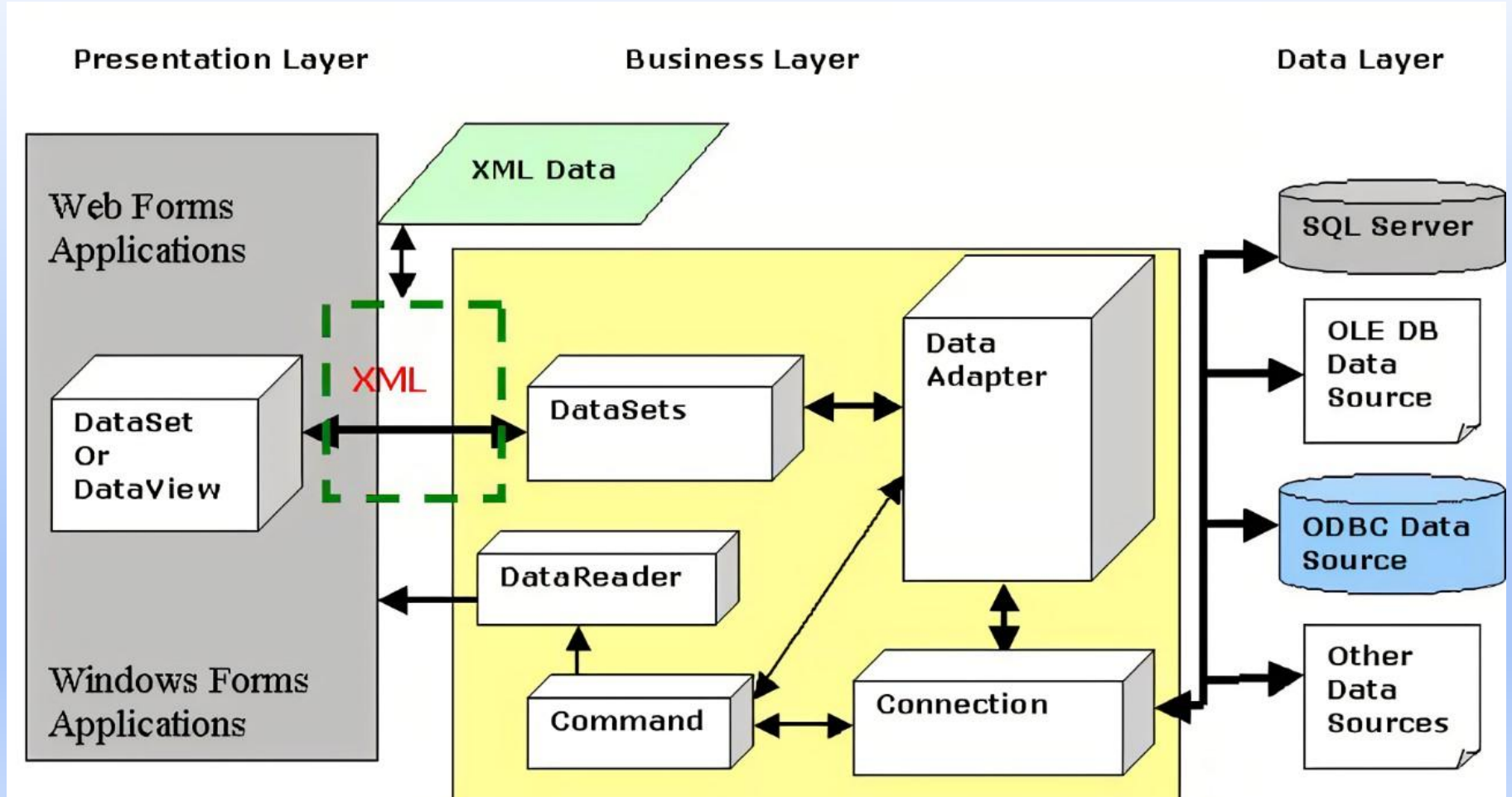


The client sends requests indirectly to the adaptee by using the adapter’s target interface.



The adapter translates the request sent by the client class into a request that the adaptee class is expecting.

Database Connector



How is Facade different from the Adapter design pattern?

How is Facade different from the Adapter design pattern?

In the Adapter pattern, you are trying to alter an interface so that your clients do not see any difference between the interfaces.

By contrast, the Facade pattern simplifies the interface.

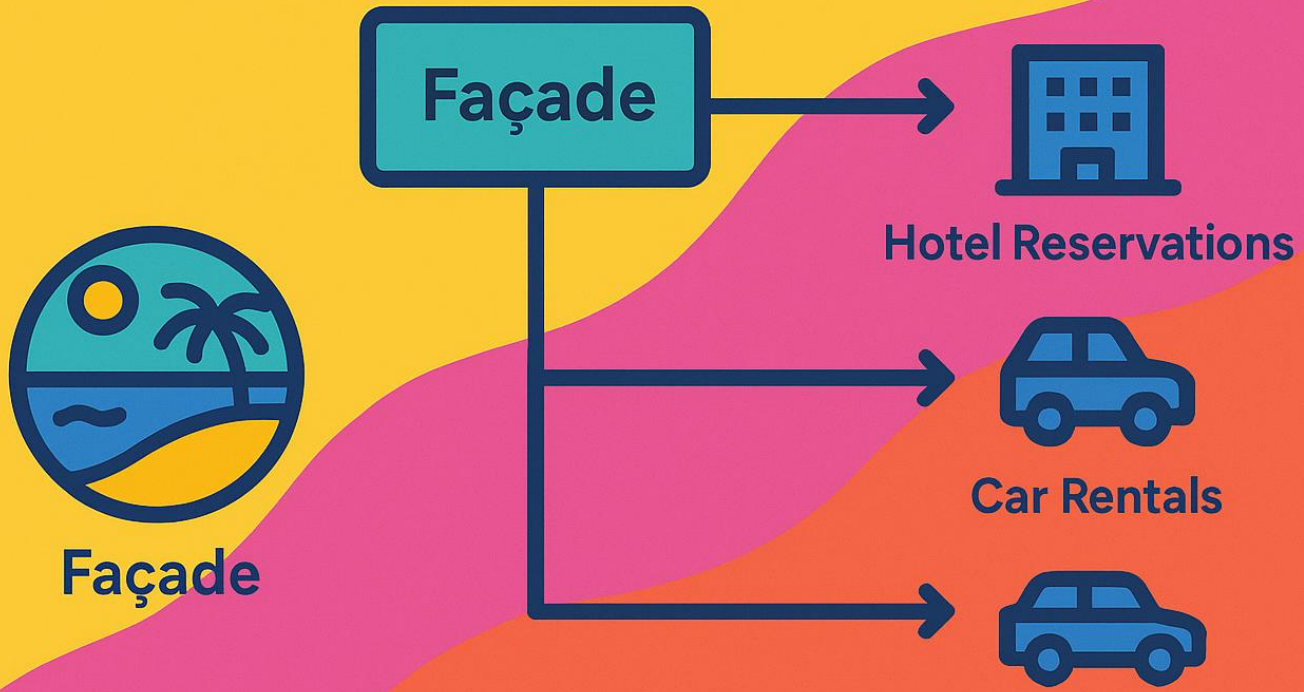
It presents the client with a simple interface to interact with (instead of a complex subsystem).

Adapter design needs more code. Do you agree?

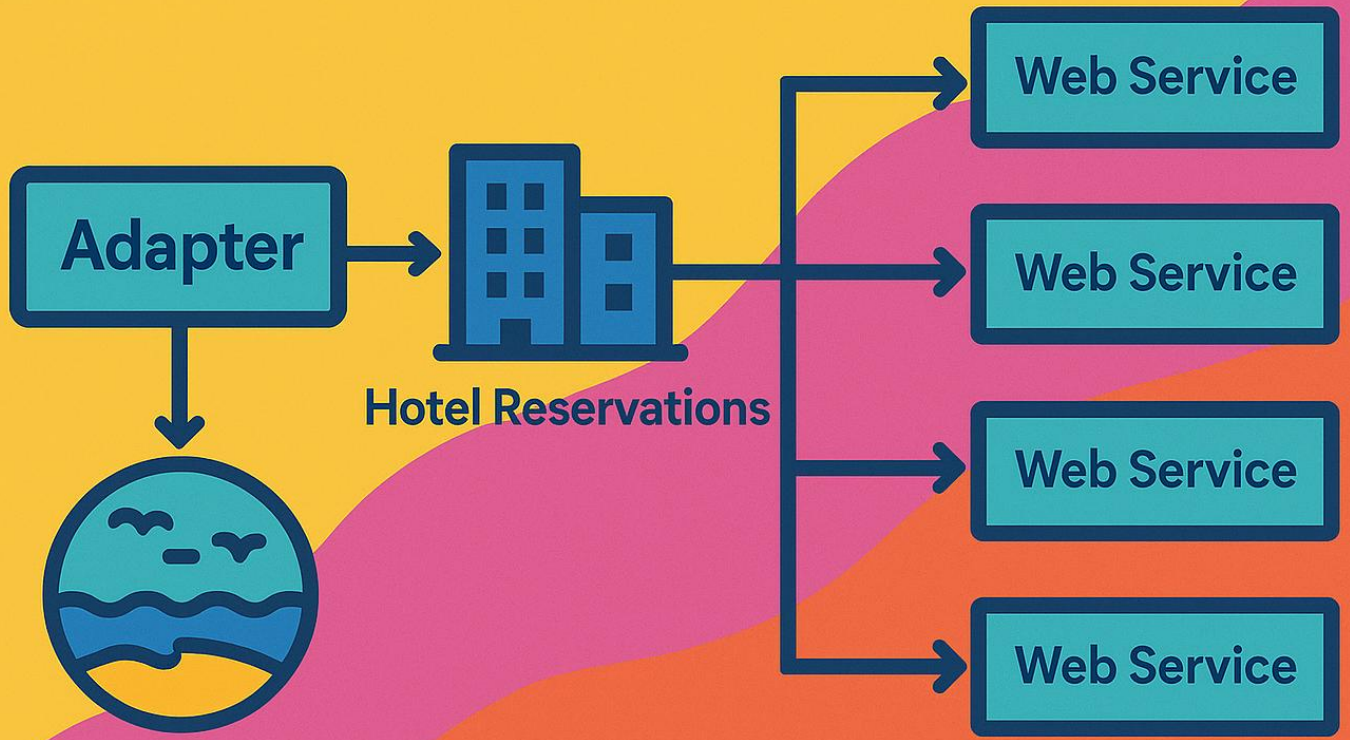
You may need to write some additional code.
However, the payoff is great, particularly for those legacy systems that cannot be changed but you still want to use them for their stability

Re

How can the Façade design pattern be used to streamline the process of booking a vacation package that involves interactions with multiple services, like Flight booking, hotel reservations, and car rentals?



How can the Adapter design pattern be used to make hotel reservations compatible with multiple web services?



Case	Example
Home Theater System Controller	You bought a new sound system that uses Bluetooth commands, but your existing Home Theater Controller only supports infrared (IR).
Smart Home Automation	Your smart home hub works with Wi-Fi-based devices, but you purchased Zigbee-enabled light bulbs.
Banking System	The new loan approval module is built on a different API format than the old banking software.
Video Game Engine	A game engine uses its own graphics API, but you want to use a third-party physics engine with a different coordinate system and method naming.
Restaurant Online Ordering	The restaurant's online ordering platform integrates with one type of payment gateway, but you now want to support PayPal.
Hospital Management System	A new insurance verification service uses XML-based requests, but your hospital system uses JSON.
E-Commerce Checkout Process	Your e-commerce platform integrates with a shipping provider's API, but you switch to a new courier whose API uses different parameters and naming conventions.

Scenario	Facade Design Pattern – Simplifies interaction with multiple subsystems	Adapter Design Pattern – Converts one interface into another compatible interface
1. Home Theater System Controller	HomeTheaterFacade with watchMovie() method that turns on TV, sound, DVD, dims lights, and closes curtains in one step.	AudioSystemAdapter converts IR commands from the existing controller into Bluetooth commands for a new sound system.
2. Smart Home Automation	SmartHomeFacade with activateAwayMode() that locks doors, turns off lights, lowers thermostat, and arms security.	ZigbeeToWiFiAdapter converts hub's Wi-Fi commands into Zigbee so incompatible smart bulbs work.
3. Banking System	LoanProcessingFacade with approveLoan() that runs credit checks, account verification, and document processing.	LoanModuleAdapter converts old banking system requests into the new loan module's API format.
4. Video Game Engine	GameEngineFacade with startGame() that loads assets, initializes physics, sets UI, and connects to server.	PhysicsEngineAdapter maps game engine calls to a third-party physics engine with a different coordinate system/API.
5. Restaurant Online Ordering	OrderFacade with placeOrder() that handles menu check, payment, kitchen notification, and delivery dispatch.	PayPalAdapter lets the ordering platform use PayPal even though it was built for another payment gateway.
6. Hospital Management System	AdmissionFacade with admitPatient() that checks insurance, assigns a doctor, creates a file, and alerts the ward.	InsuranceServiceAdapter converts hospital's JSON requests into XML for a new insurance service.
7. E-Commerce Checkout Process	CheckoutFacade with checkout() that verifies items, applies discounts, processes payment, and triggers shipping.	ShippingServiceAdapter converts platform's shipping calls to match a new courier's API format.